本周工作

亚多普勒冷却

物理机制

前提：光抽运时间远大于辐射寿命，需要考虑空间偏振梯度
激光组态：“Lin \bot Lin”,反向传播相互垂直，即每隔$\lambda/4$偏振垂直

$$E(z) = E_0(\hat{e}_x e^{(ikz-\omega t)} + \hat{e}_y e^{i(-kz-\omega t)})$$

分解为圆偏振分量：

取约定
$\hat{e}_\pm = (\hat{e}_x \mp i\hat{e}_y)/\sqrt{2}$，则

$$E_+(z) = \frac{E_0}{\sqrt{2}} e^{-i\omega t}\left(e^{ikz} + ie^{-ikz}\right) = E_0 e^{-i\omega t} e^{i\pi/4} \cos\left(kz - \frac{\pi}{4}\right)$$
$$E_-(z) = \frac{E_0}{\sqrt{2}} e^{-i\omega t}\left(e^{ikz} - ie^{-ikz}\right) = E_0 e^{-i\omega t} e^{-i\pi/4} \cos\left(kz + \frac{\pi}{4}\right)$$

原子能级：$J_g = 1/2 \to J_e = 3/2$，前者有两个简并磁量子数态，后者有四个简并磁量子数态

- 由 Clebsch-Gordan 系数平方给出自发辐射分支比:

G\E	$\lvert e,+3/2\rangle$	$\lvert e,+1/2\rangle$	$\lvert e,-1/2\rangle$	$\lvert e,-3/2\rangle$
$\lvert g+\rangle$	1	$2/3$	$1/3$	0
$\lvert g-\rangle$	0	$1/3$	$2/3$	1

- 以及偶极矩阵元：

令约化矩阵元为 $d \equiv \langle J_e = 3/2 \| \mathbf{d} \| J_g = 1/2\rangle$,
则在上面的 $\sigma^\pm$ 分解下，电偶极耦合写成

$$\langle e, m_e | \mathbf{d}\cdot\mathbf{E} | g, m_g\rangle = d\, C^{3/2,m_e}_{1/2,m_g;1,q}\, E_q(z), \quad q = m_e - m_g = \pm 1$$

定义

$$\Omega_0 \equiv \frac{dE_0}{\hbar}$$

则矩阵元可直接写成具体形式（含空间依赖）：

对应的非零耦合（Clebsch-Gordan 系数）为：

跃迁	偏振	$C^{3/2,m_e}_{1/2,m_g;1,q}$	偶极矩阵元
$\lvert g,+1/2\rangle \to \lvert e,+3/2\rangle$	σ^+	1	$\hbar\,\Omega_0\, e^{-i\omega t} e^{i\pi/4} \cos\!\left(kz - \frac{\pi}{4}\right)$
$\lvert g,-1/2\rangle \to \lvert e,+1/2\rangle$	σ^+	$1/\sqrt{3}$	$\frac{\hbar\,\Omega_0}{\sqrt{3}}\, e^{-i\omega t} e^{i\pi/4} \cos\!\left(kz - \frac{\pi}{4}\right)$
$\lvert g,+1/2\rangle \to \lvert e,-1/2\rangle$	σ^-	$1/\sqrt{3}$	$\frac{\hbar\,\Omega_0}{\sqrt{3}}\, e^{-i\omega t} e^{-i\pi/4} \cos\!\left(kz + \frac{\pi}{4}\right)$
$\lvert g,-1/2\rangle \to \lvert e,-3/2\rangle$	σ^-	1	$\hbar\,\Omega_0\, e^{-i\omega t} e^{-i\pi/4} \cos\!\left(kz + \frac{\pi}{4}\right)$

在激光旋转参考系中（并采用 RWA）下，设失谐

$$\Delta = \omega - \omega_{at}$$

并定义

$$\Omega_+(z) = \Omega_0 e^{i\pi/4}\cos\Big(kz - \frac{\pi}{4}\Big), \qquad \Omega_-(z) = \Omega_0 e^{-i\pi/4}\cos\Big(kz + \frac{\pi}{4}\Big)$$

则相互作用哈密顿量写为

$$H_{\mathrm{rot}}(z) = -\hbar\Delta\sum_{m_e}|e,m_e\rangle\langle e,m_e| - \hbar\Big[\Omega_+(z)|e,+3/2\rangle\langle g,+1/2| + \frac{\Omega_+(z)}{\sqrt{3}}|e,+1/2\rangle\langle g,-1/2| + \frac{\Omega_-(z)}{\sqrt{3}}|e,-1/2\rangle\langle g,+1/2| + \Omega_-(z)|e,-3/$$

若取基底顺序

$$\{|g,+1/2\rangle, |g,-1/2\rangle, |e,+3/2\rangle, |e,+1/2\rangle, |e,-1/2\rangle, |e,-3/2\rangle\}$$

对于 MCWF 模拟，再加入自发辐射导致的非厄米项。若激发态总衰减率为 Γ，则有效哈密顿量为

$$H_{\mathrm{eff}}(z) = H_{\mathrm{rot}}(z) - \frac{i\hbar\Gamma}{2}\sum_{m_e}|e,m_e\rangle\langle e,m_e|$$

在同一基底下，其矩阵形式为

$$H_{\mathrm{eff}}(z) = \hbar\begin{pmatrix} 0 & 0 & -\Omega_+^*(z) & 0 & -\Omega_-^*(z)/\sqrt{3} & 0 \\ 0 & 0 & 0 & -\Omega_+^*(z)/\sqrt{3} & 0 & -\Omega_-^*(z) \\ -\Omega_+(z) & 0 & -\Delta - i\Gamma/2 & 0 & 0 & 0 \\ 0 & -\Omega_+(z)/\sqrt{3} & 0 & -\Delta - i\Gamma/2 & 0 & 0 \\ -\Omega_-(z)/\sqrt{3} & 0 & 0 & 0 & -\Delta - i\Gamma/2 & 0 \\ 0 & -\Omega_-(z) & 0 & 0 & 0 & -\Delta - i\Gamma/2 \end{pmatrix}$$

在动量格点中，$cos(kz)$ 可以表示为最近邻耦合，方法在之前已经使用过，不再赘述。

极限温度估计与模拟结果

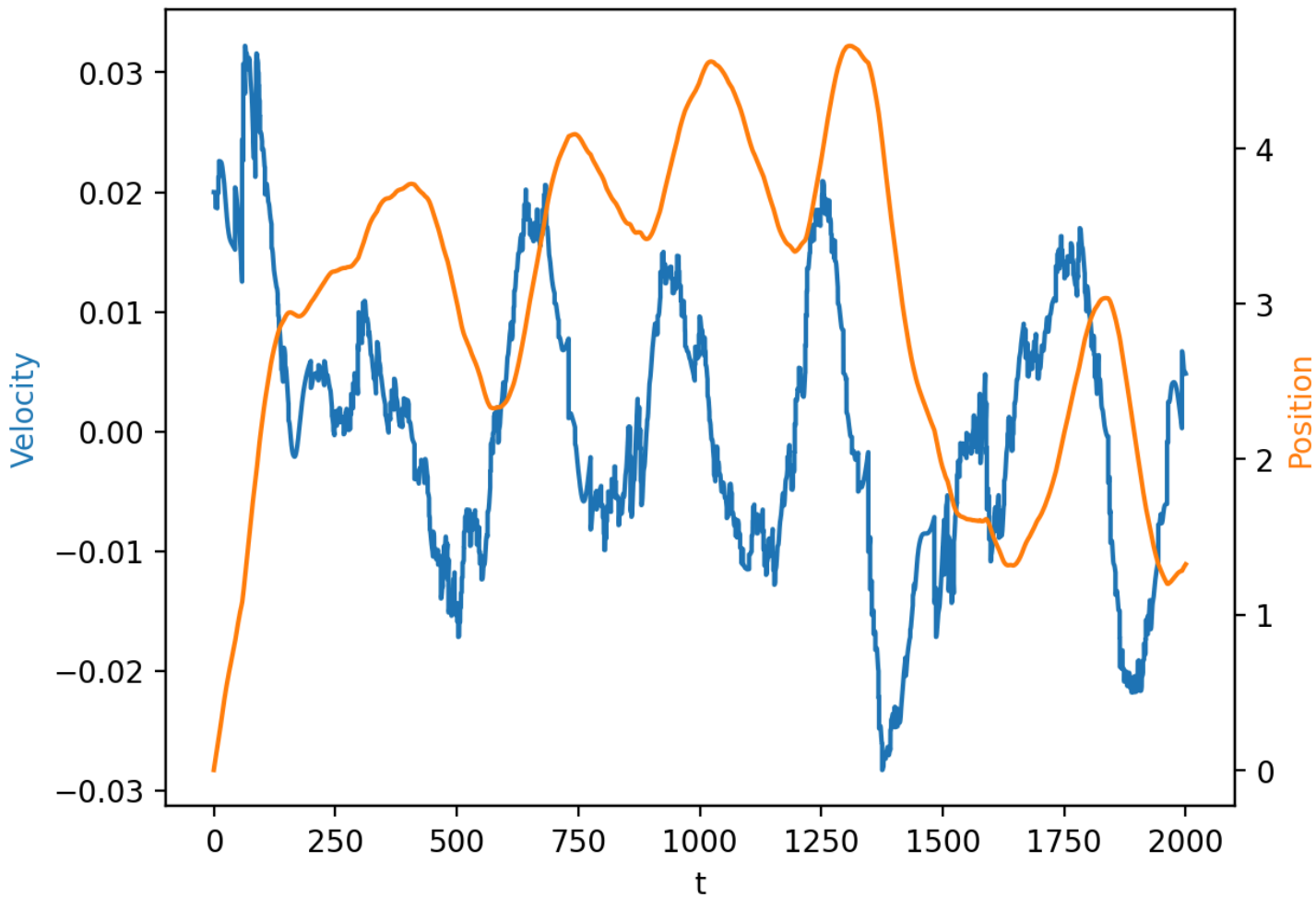
一维模拟中，可以估计Δp的展宽。

> 以下模拟均有$\hbar = 1$ and $M = 800.0$

多普勒冷却

$$< \frac{p^2}{2M} >= \frac{1}{2}k_B T_m = \frac{\hbar\Gamma}{4}$$

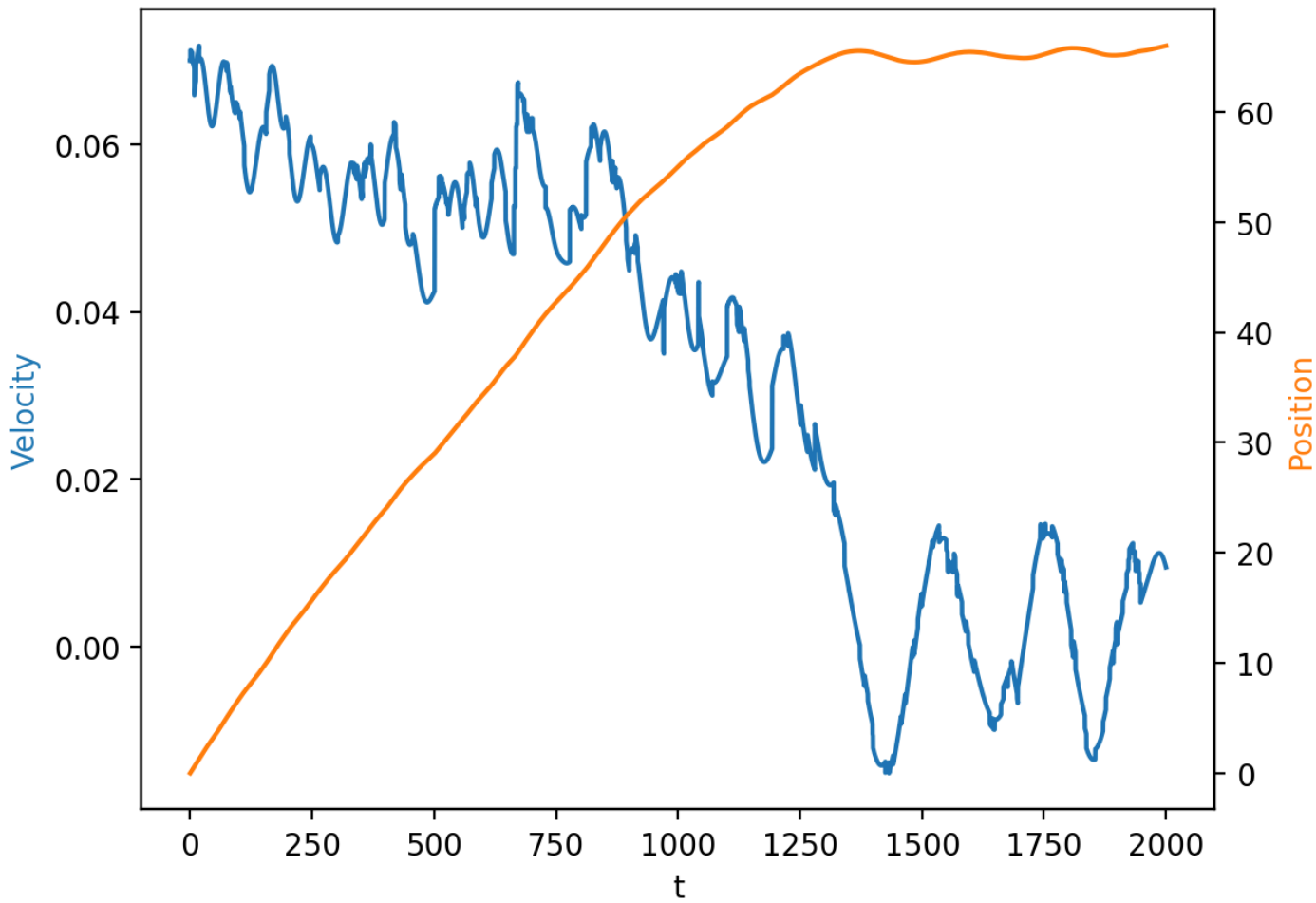
在我们的模拟中有$\Delta v \approx \sqrt{0.5\Gamma/M} = 0.025$
对照模拟结果



亚多普勒冷却

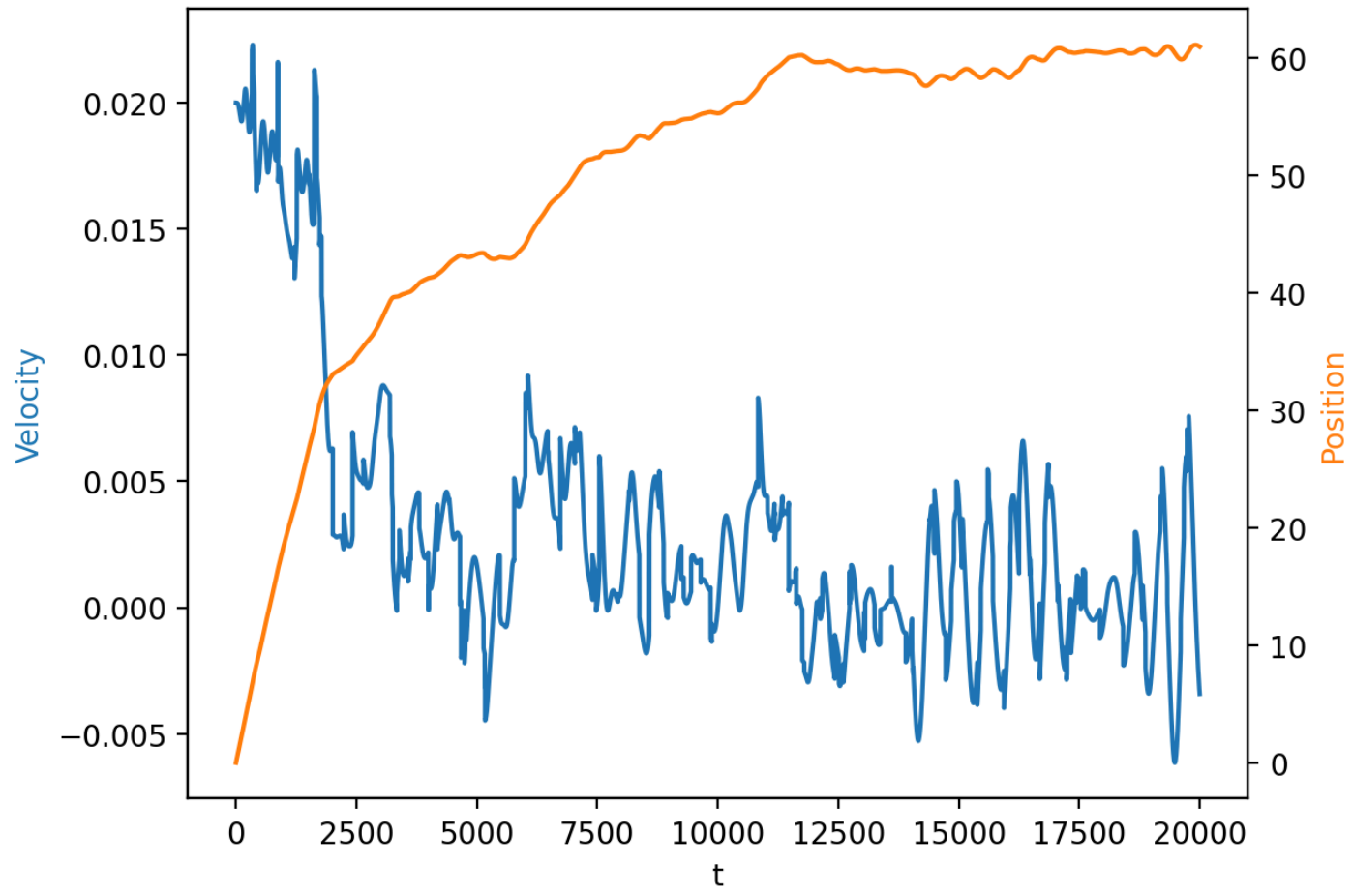
$k_B T \approx \frac{\hbar\Omega^2|\delta|}{\delta^2+\Gamma^2}$ thus $\Delta v = \sqrt{\frac{\Omega^2|\delta|}{\delta^2+\Gamma^2}/M}$

- 初次模拟
  $\delta = -5.0\,\Omega = 2.0\,\Gamma = 1.0$ and $\Delta v \approx 0.031$



验证了降温的可行性，但没有突出冷却优势

- 补充模拟
  $\delta = -10.0\ \Omega = 1.0\ \Gamma = 1.0$ and $\Delta v \approx 0.011$



展宽基本符合理论预测，而且实现了长时间的束缚

代码

```
import numpy as np
from lib import *

# parameters, and It's needless to say hbar = 1 kL = 1
delta = -10.0
Omega0 = 1.0
Gamma = 1.0
# time parameters
N = 50000000
dt = 0.0004
t = np.linspace(0.0, N * dt, N + 1)
# initial conditions
M = 800.0
v0 = 0.02

n_max = int(M*v0) + 200 # larger than M*v0
psi0 = np.zeros((6, 2*n_max+1), dtype=complex)
psi0[0, int(n_max + M*v0)] = 1.0
p_grid = np.linspace(-n_max, n_max, 2*n_max+1)

JUMP_CHANNELS = (
    (2, 0, 1.0),
    (3, 0, 2.0 / 3.0),
    (3, 1, 1.0 / 3.0),
    (4, 0, 1.0 / 3.0),
    (4, 1, 2.0 / 3.0),
    (5, 1, 1.0),
)

class H_eff:
    def __init__(self, p_grid: np.ndarray, M: float, delta: float, Gamma: float, Omega0: float):
        p = np.asarray(p_grid, dtype=float)
        if p.ndim != 1:
            raise ValueError("p_grid 必须是一维数组")

        self.p = p
        self.M = float(M)
        self.delta = float(delta)
        self.Gamma = float(Gamma)
        self.Omega0 = float(Omega0)

        self.T = (p * p) / (2.0 * self.M)
        self.e_diag = self.T + ((-self.delta) - 0.5j * self.Gamma)
        self.c1 = 0.5 * self.Omega0
        self.c3 = self.c1 / np.sqrt(3.0)

    def apply(self, psi: np.ndarray) -> np.ndarray:
        psi_arr = np.asarray(psi, dtype=np.complex128)
        expected_shape = (6, self.p.shape[0])
        if psi_arr.shape != expected_shape:
            raise ValueError(f"psi shape 应为 {expected_shape}，实际得到 {psi_arr.shape}")

        gp = psi_arr[0]
        gm = psi_arr[1]
        e32 = psi_arr[2]
        e12 = psi_arr[3]
        em12 = psi_arr[4]
        em32 = psi_arr[5]

        c1 = self.c1
        c3 = self.c3

        out = np.empty_like(psi_arr)
```

```
        out[0] = self.T * gp
        out[0, :-1] += -c1 * e32[1:] - c3 * em12[1:]
        out[0, 1:] += 1j * c1 * e32[:-1] - 1j * c3 * em12[:-1]

        out[1] = self.T * gm
        out[1, :-1] += -c3 * e12[1:] - c1 * em32[1:]
        out[1, 1:] += 1j * c3 * e12[:-1] - 1j * c1 * em32[:-1]

        out[2] = self.e_diag * e32
        out[2, 1:] += -c1 * gp[:-1]
        out[2, :-1] += -1j * c1 * gp[1:]

        out[3] = self.e_diag * e12
        out[3, 1:] += -c3 * gm[:-1]
        out[3, :-1] += -1j * c3 * gm[1:]

        out[4] = self.e_diag * em12
        out[4, 1:] += -c3 * gp[:-1]
        out[4, :-1] += 1j * c3 * gp[1:]

        out[5] = self.e_diag * em32
        out[5, 1:] += -c1 * gm[:-1]
        out[5, :-1] += 1j * c1 * gm[1:]

        return out

def mcwf_step(ket: Quantumket, rng: np.random.Generator):
    dp = 1.0 - ket.norm2()
    if dp < 0.0:
        dp = 0.0

    if rng.random() >= dp:
        return ket.normalize(), False

    weights = np.zeros(len(JUMP_CHANNELS), dtype=float)
    for i, (e_idx, _g_idx, branch) in enumerate(JUMP_CHANNELS):
        weights[i] = branch * float(np.vdot(ket.data[e_idx], ket.data[e_idx]).real)
    weights = weights / np.sum(weights)
    channel_index = int(rng.choice(len(JUMP_CHANNELS), p=weights))

    e_idx, g_idx, _ = JUMP_CHANNELS[channel_index]
    source = ket.data[e_idx]
    jumped = np.zeros_like(ket.data)

    if rng.random() < 0.5:
        jumped[g_idx, 1:] = source[:-1]
    else:
        jumped[g_idx, :-1] = source[1:]

    ket = Quantumket._fast_from(ket, jumped).normalize()
    return ket, True

def main():
    heff = H_eff(p_grid, M, delta, Gamma, Omega0)
    ket = Quantumket(
        psi0,
        ["g+", "g-", "e+3/2", "e+1/2", "e-1/2", "e-3/2"],
        tuple([p_grid]),
        "p",
    )

    v_history = np.zeros(N + 1, dtype=float)
    x_history = np.zeros(N + 1, dtype=float)
    rhoe_total_history = np.zeros(N + 1, dtype=float)
```

```
    rng = np.random.default_rng()

    v_history[0] = ket.expect() / M
    x_history[0] = 0.0
    rhoe_total_history[0] = (
        ket.population("e+3/2") + ket.population("e+1/2") + ket.population("e-1/2") + ket.population("e-3/2")
    )

    for step in range(N):
        ket = RKstepper(ket, heff, dt)
        ket, jumped = mcwf_step(ket, rng)

        v_history[step + 1] = ket.expect() / M
        x_history[step + 1] = x_history[step] + v_history[step] * dt
        rhoe_total_history[step + 1] = (
            ket.population("e+3/2") + ket.population("e+1/2") + ket.population("e-1/2") + ket.population("e-3/2")
        )

        if step % 1000 == 0:
            print(f"step: {step} v: {v_history[step + 1]:.4f}, total excited: {rhoe_total_history[step + 1]:.4f}")

    plot_dual_axis(t, v_history, x_history, "Velocity", "Position", save_path="subdropping_velocity_position.png")
    plot_populations(t, rhoe_total_history, labels=["e"], title="Excited State Population", save_path="subdropping_excited_populatio

if __name__ == "__main__":
    main()
```

亚反冲冷却

新建文件夹